Yang Zhao

857-225-1826 • yangzhao200124@gmail.com • linkedin.com/in/yang-zhao-48b12431a • github.com/YZhao-prog

EDUCATION

Northeastern University — Boston, MA
Master of Science in Information Systems, GPA: 3.8/4.0 — Sep. 2024 – Dec. 2026 (Expected)
Coursework: Linux/UNIX Systems, Data Structures, Algorithms, Object-Oriented Design, High-Performance Computing for AI

The Chinese University of Hong Kong, Shenzhen — Shenzhen, China
Bachelor of Engineering in Computer Science and Engineering — Sep. 2020 – Jun. 2024
Coursework: Database Systems, Distributed and Parallel Computing, Operating Systems, Computer Architecture, Compilers, Computer Networks

University of California, Berkeley
Summer Session (CS 61A: Structure and Interpretation of Computer Programs), GPA: 4.0/4.0 — May 2022 – Aug. 2022

PROFESSIONAL EXPERIENCE

Splunk — San Jose, CA
Backend Software Engineer Intern — Jun. 2026 – Sep. 2026

- Designed and implemented a **disk-based cache management system** in **C++** for a distributed search platform, introducing **size-aware cache eviction** to reclaim local storage, prevent disk-full failures, and preserve search result availability.
- Built an **in-memory storage tracker**, replacing expensive **full-directory scans** with **O(1)** incremental disk usage accounting and eliminating repeated filesystem traversal.
- Engineered an artifact offloading pipeline across concurrent workers, **AWS S3**, and **PostgreSQL**, using readiness checks, conditional writes, and durable retries to sustain **500 concurrent offloads** with zero metadata inconsistencies or duplicate uploads.
- Implemented configurable **per-search disk usage guardrails** that terminate **oversized in-progress searches**, preventing a single runaway search from destabilizing the cluster.
- Instrumented the cache eviction pipeline with before/after **rollout** metrics and a dashboard tracking **cache hit/miss rate** and S3 artifact restore **latency**, validating that eviction reduced **local disk usage** and kept it near the configured limit.

Splunk — Boston, MA
Backend Software Engineer Intern — Sep. 2025 – Dec. 2025

- Built a cross-node artifact sharing system in **C++** for Splunk's distributed search platform, enabling search results to persist and be accessed across **Search Head Cluster nodes** via **AWS S3** and **REST APIs.**
- Drove a **feature-flagged rollout** of remote artifact storage from internal clusters through **customer canary** to the full production fleet.
- Optimized the artifact storage pipeline with **batching** and async uploads to sustain high-throughput workloads, enabling **horizontal scaling** by eliminating node-local state dependencies with only a **0.62%** increase in storage cost**.**
- Delivered **zero-downtime** cluster upgrades by designing a **backward-compatible storage schema**, enabling **rolling migrations** across **multi-node clusters** without service interruption.
- Created a **pytest** suite covering **artifact lifecycle** and **REST API** behavior, integrated into **CI/CD** and validated at production scale under **600K+ daily searches** with **zero failures**.

PROJECTS

SQL Database Engine (SharkDB) | Rust

- Built a **SQL database engine** from scratch with a hand-written query parser, **rule-based optimizer**, and **parallel execution engine**, achieving ~2.1× throughput over a single-threaded baseline on analytical queries.
- Implemented a **pluggable storage layer** with three interchangeable engines (in-memory, Bitcask-style log, LSM-tree), achieving ~500K QPS for storage point reads and ~200K QPS for end-to-end SQL point lookups.
- Designed an **MVCC transaction system** with **snapshot isolation** and optimistic concurrency control, validated against dirty, non-repeatable, and phantom read anomalies with strong consistency guarantees.
- Wrote a custom **LSM-tree storage engine** in Rust with memtables, SSTables, leveled compaction, and Bloom filters for efficient range scans, with WAL-based crash recovery ~16× faster than full-log replay.

Distributed Key-Value Store with Raft Consensus | Go, RPC

- Implemented **Raft consensus protocol** with leader election, log replication, persistence, snapshot compaction, and crash recovery, ensuring correct operation under network partitions.
- Built linearizable Key/Value service on **Raft**, supporting concurrent clients with request deduplication and at-most-once semantics, and validated correctness using linearizability testing.
- Developed **sharded Key/Value store** with replicated Shard Controller, supporting dynamic shard migration and rebalancing across Raft-backed replica groups while preserving consistency.

TECHNICAL SKILLS

Languages: C++, Rust, Go, Java, Python, SQL, JavaScript/TypeScript, HTML/CSS
Backend & Web: Spring Boot, Django, Node.js/Express, gRPC, Protocol Buffers, REST APIs, React, Next.js, Angular
GPU & Parallel Computing: CUDA, OpenMP, OpenACC, MPI, SIMD
Databases & Messaging: MySQL, PostgreSQL, MongoDB, Redis, RocksDB, Elasticsearch, Kafka, RabbitMQ
Cloud & Infrastructure: AWS (EC2, S3, Lambda, RDS), Docker, Kubernetes, Helm, Jenkins, Git, Linux, Bash, CMake, CI/CD

Observability: Splunk, OpenTelemetry, Datadog, Prometheus, Grafana